

[3번] 백엔드 조사 루프 및 대시보드 연동 (INV, STA, CHT 도메인)

≡ 진행상황

진행중

[3번: 효선, 정윤] - 백엔드 조사 루프 및 대시보드 연동 (INV, STA, CHT 도메인)

- 담당 기능 ID: INV-01, INV-02, INV-03, STA-01, STA-02, CHT-01
- 사용할 데이터: (입력용) `data/processed/llm_struct_test_results.jsonl`, (조회용) `data/processed/hk_electrical_recent_full.csv`
- To-Do:
 - 효선: 쿼리 템플릿 (INV-01): 1.6만 건 CSV를 바탕으로 AI가 안전하게 데이터 통계를 조회할 수 있는 파이썬 조회 함수(연식 분포, 월별 추이 등 5~8종) 작성.
 - 효선: 조사 루프 (INV-02, 03): AI가 스스로 가설 수립 → 쿼리 템플릿 실행 → 결과에 따라 가설 유지/기각을 판단하는 반복(`while`) 로직 구현 및 근거 부족 시 정지하는 장치 마련.
 - 정윤: 상태 추적 DB (STA-01, 02): 특정 차종의 결함이 '신규'인지 '기존 리콜 후 재발'인지 기록하고 추적하는 상태 DB(SQLite 수준) 뼈대 구축.
 - 정윤: 채팅 연동 (CHT-01): 질의응답 시나리오(질문 → 구조화 → 조사 → 검수 → 답변) 파이프라인 연결.

INV-01/02/03 설계 설명 (효선)

"조사(③)" 단계 — 급증 구간에서 LLM이 가설을 세우고, 데이터로 검증하고, 유지/기각을 판단하는 부분. 이 문서는 팀원들이 "이게 왜 이렇게 짜여있는지" 이해할 수 있게 설계 의도부터 설명함.

1. 왜 이렇게 나눴는가 — 두 파일, 한 가지 원칙

```
query_templates.py      INV-01: "무엇을 조회할 수 있는가" (도구 정의)
investigation_loop_v2.py INV-02/03: "언제 어떤 도구를 쓸지, 결과를 어떻게 판단할지" (루프)
```

핵심 원칙 (팀 가이드라인 4번: "계산·검색·검증은 도구와 분리, LLM은 사고·판단")

LLM은 절대 숫자를 계산하지 않는다. LLM이 하는 일은 딱 세 가지뿐:

① 정찰 결과를 보고 가설 문장 쓰기 ② 다음에 어느 함수를 부를지 이름으로 고르기 ③ 결과를 보고 유지/기각/보류 판단하기

이 원칙 때문에 `query_templates.py`의 모든 함수는:

- pandas 집계만 함 (LLM이 이 안에 관여 안 함)
- 항상 dict를 반환 (LLM이 그대로 읽을 수 있게)
- 0건이어도 예외 안 던지고 `{"status": "no_data"}` 로 정직하게 알림

2. INV-01 (`query_templates.py`) — 쿼리 함수 8종

왜 "자유 코드 생성"이 아니라 "함수 목록에서 선택"인가

LLM에게 "판다스 코드를 직접 짜서 실행해"라고 하면 빠르긴 한데, 버그·보안·비결정성을 통제할 수가 없음(가이드라인 문서에 명시된 이유). 그래서 **미리 만들어둔 함수 8개의 "이름"만 LLM한테 보여주고, 그중에서 고르게 함**:

```
TOOL_REGISTRY = {
    "get_symptom_distribution": {...},
    "get_model_year_breakdown": {...},
    ...
}
```

LLM이 받는 건 함수 코드가 아니라 이 dict의 `when_to_use` 설명뿐 — "부품 카테고리 안에서 대표 증상이 뭔지 먼저 파악할 때" 같은 자연어 안내.

8개 함수가 왜 이 구성인가

함수	역할	왜 필요한가
<code>get_symptom_distribution</code>	증상 분포	조사 루프의 사실상 1번 타자 — "이 부품에서 뭐가 문제인지" 먼저 봐야 시작 가능
<code>get_model_year_breakdown</code>	차종×연식 교차표	가장 중요한 함수. "2025년식 투싼"처럼 두 조건이 결합된 가설은, 연식별 분포·차종별 분포를 각각 따로 봐서는 검증 불가능함 (예: 투싼 전체는 안 늘었는데 2025년식만 늘었을 수 있음) → 반드시 교차표가 있어야 조합 가설을 확인 가능
<code>get_year_distribution</code>	연식만	차종 무관 트렌드 (계절/연식 자체 이슈 확인용)
<code>get_model_distribution</code>	차종만	연식 무관, 특정 차종 고유 문제인지
<code>get_monthly_trend</code>	월별 추이	계절성(여름 에어컨 등)과 진짜 급증을 구분하는 용도
<code>get_severity_breakdown</code>	심각도 분포	우선순위 매기는 용도
<code>check_recall_match</code>	리콜 이력 대조	"이미 리콜된 적 있나" 확인 → STA 트랙과 겹치는 개념이지만, INV는 즉석 조사 중 참고용으로 가볍게 확인
<code>get_us_kr_gap</code>	한·미 시차	국내 미조치 여부 확인 (프로젝트 셀링포인트)

데이터 흐름: `load_enriched_df()`

```
raw = pd.read_csv(RAW_CSV_PATH) # 원본 신고 (16,964건, model/model_year 포함)
struct = pd.read_json(STRUCT_JSONL_PATH, lines=True) # STR 산출물 (odino, part_category, symptoms, severity)
df = struct.merge(raw[...], on="odino", how="left") # odino로 조인
```

왜 조사 루프 시작 시 1번만 호출하고 캐싱하나 — 매 쿼리마다 파일을 다시 읽으면 느리고, 조사 하나에 최대 9번(가설 3개×쿼리 3개)까지 쿼리가 돌 수 있어서 캐싱이 필수.

COLUMN_MAP 이 왜 있는가

원본 CSV 컬럼명이 실제 확인 전까지 불확실했어서, 하드코딩 대신 딕셔너리로 빼둠 — 나중에 실제 스키마 확인되면 이 dict만 고치면 나머지 코드는 안 건드려도 되게 설계. (오늘 실제로 이 덕분에 mock → 실데이터 전환이 값 3개만 바꾸는 걸로 끝났음)

3. INV-02/03 (investigation_loop_v2.py) — 조사 루프

전체 흐름 (ReAct 패턴)

```
0단계: 정찰(scouting) - 코드가 강제 실행 (LLM 관여 없음)
↓
1단계: LLM이 정찰 결과 보고 가설 생성 (generate_hypothesis)
↓
2단계: LLM이 추가 쿼리 선택 or 결론 (select_query_or_conclude) - 최대 3회 반복
↓
3단계: LLM이 최종 판정 (judge_hypothesis) → RETAINED / REJECTED / HOLD
↓ (REJECTED면)
가설 다시 생성 (최대 3개까지 시도)
```

왜 "정찰"이 코드로 강제되는가

```
SCOUTING_QUERIES = ["get_symptom_distribution", "get_model_year_breakdown"]
```

LLM이 아무 근거 없이 첫 가설부터 세우면 "느낌"으로 판단하게 됨. 그래서 가설을 세우기 전에 무조건 이 2개 쿼리부터 실행해서 실제 숫자를 보여주고 시작하게 만들 — "실제 숫자 없이 가설 금지"라는 설계 원칙.

[이식 1] 조건 완화 4단계 사다리 (RELAXATION_LADDER)

```
RELAXATION_LADDER = [
    {"label": "exact", "drop": []},
    {"label": "drop_text_keyword", "drop": ["symptom"]},
    {"label": "drop_year", "drop": ["symptom", "model_year"]},
    {"label": "drop_model", "drop": ["symptom", "model_year", "model"]},
]
```

쿼리 결과가 0건이면, "증상 조건 빼고 재시도 → 그래도 없으면 연식도 빼고 → 그래도 없으면 차종도 빼고" 순서로 자동으로 조건을 넓혀감. 왜 LLM한테 매번 "완화할까요?"를 안 묻는가 — 이걸 비용/시간 낭비이고, "0건일 때 어떻게 복구할지"는 사람이 봐도 뻔한 절차라 코드가 결정론적으로 처리. 대신 유지/기각 최종 판단은 절대 코드가 안 함 — 숫자 임계값으로 자동 확정하는 걸 의도적으로 배제(팀원 코드 리뷰하면서 이 부분은 안 가져옴).

[이식 2] 한국어-영어 키워드 매핑

STR 산출물의 symptoms 가 한국어인데, 원문(CDESCR)은 영어라서 텍스트 검색을 하려면 변환이 필요함:

```
"계기판": ["dashboard", "instrument", "cluster", "display", "screen", "speedometer"]
```

[이식 3] flatten_result() / save_investigation_log()

조사 결과를 Judge(④)와 대시보드가 파싱하기 쉬운 고정 CSV 스키마로 저장. INV가 자기 조사 기록을 다른 트랙이 읽을 수 있게 표준화해두는 부분.

[이식 4] `investigate_ad_hoc()` — CHT-01과의 연결점

```
def investigate_ad_hoc(question_parsed, df, llm_call):
```

급증 감지(①)가 아니라 사용자가 직접 질문했을 때도 똑같은 조사 루프를 재사용하기 위한 진입점. STR-05(질문 이해 모듈)가 뽑아준 `{model, model_year, symptom}` 을 가짜 STR 레코드 1건짜리 배치로 감싸서, `run_investigation_loop()` 를 그대로 태움. "상시 모니터링과 질의응답이 동일 파이프라인을 쓴다"는 설계 원칙(주제 개요 4-2)을 구현한 부분.

왜 `is_retained` 가 3분기(`True` / `False` / `None`)인가

`True` → RETAINED (가설이 데이터로 뒷받침됨)
`False` → REJECTED (가설이 반박됨, 다음 가설 시도)
`None` → HOLD (근거 부족, 보류 — "모르면 모른다" 원칙)

이건 팀 가이드라인의 "모름 3중 장치" 중 하나("Self-Consistency 불일치/증거 부족 → 모름")를 INV 레벨에서 미리 반영해둔 것. 이 3분기 형식은 ④판정(영운님 JDG)이 그대로 이어받아서 다수결 처리하는 인터페이스로 씬.

4. 오늘 실행에서 실제로 확인된 것 (검증 로그)

질문: "2025년식 투싼 계기판 꺼짐 최근 6개월 시그널 있어?"

0단계 정찰: `get_symptom_distribution`, `get_model_year_breakdown` 실행
1단계 (Gemini): 가설 생성 — 정찰 결과 없음을 근거로 "가설 세울 수 없다"고 정직하게 응답
2단계 (Gemini): `conclude` 선택 (추가 쿼리 없이 바로 결론)
3단계 (Gemini): `judge_hypothesis` → RETAINED
→ 판정 근거에 실제 `status:no_data` 값을 인용 (숫자 지어내지 않음)

→ "근거 없으면 없다고 말한다"는 설계 의도가 실제로 작동하는 걸 확인. mock 데이터라 결과 자체는 재미없었지만(진짜 급증 케이스가 아니었음), 로직이 의도대로 도는 건 증명됨.

5. 알려진 한계 / 아직 못 본 것

- 여러 가설(최대 3개) 순환 — 이번엔 1번째에서 바로 끝나서 "기각 후 재시도" 경로 미확인
- LLM이 정찰 후 추가 쿼리를 실제로 선택하는 순간 — 이번엔 바로 `conclude`로 감
- `RELAXATION_LADDER` 실제 작동 — 이번엔 정찰 단계라 완화 사다리를 안 탐 (완화는 `select_query_or_conclude` 에서 고른 쿼리가 0건일 때만 작동)
- 오늘 발견된 버그: `get_model_year_breakdown` 등에서 join 실패(`model_year` 전부 NaN)로 인한 groupby 결과 없음 케이스 방어 부족 → 수정 중

INV 트랙(효선) 작업 현황 — 2026-07-15 (최종 업데이트)

오늘 하루 동안 mock 데이터 검증 → 실제 Git 데이터 발굴 → 진짜 리콜/EV9 시나리오 실증까지 완료. 아래는 최종 결과 기준으로 정리.

오늘 한 일 요약

INV-01(`query_templates.py`)·INV-02/03(`investigation_loop_v2.py`)을 실제 Gemini API와 연결해서 end-to-end 실행 검증하고, STA 트랙(정윤)을 실제 원본 데이터 16,964건 + 실제 NHTSA 리콜 264건으로 완주시켜 STA-02(재발)·RECALLED·DORMANT 상태를 실제 사례(EV9 계기판 리콜)로 처음 실증함.

✓ 최종 검증 완료된 것

1. STA 파이프라인 — 실제 데이터로 end-to-end 완주

```
python sta_run_pipeline.py --csv "hk_electrical_recent_full_REAL.csv" --jsonl "llm_struct_test_results.jsonl"
```

[STA LOAD CSV] source_rows=16,964 (실제 원본, mock 아님)
[STA RECALL] rcl573_components_normalized.csv → recall_records 264건 (실제 NHTSA 리콜, MODEL_TO_MAKE 매핑 완료 후 WARN 0건)
[STA CLASSIFY] STA-02=32건, STA-01=17건
[STA SIGNAL] 상태 분포 = NEW 15 / DORMANT 3 / RECALLED 1

실제 사례로 확인된 것:

- KIA EV9 2024 INSTRUMENT_CLUSTER 신고 8건 → 실제 리콜 24V757000 (2024-2025 EV9 계기판 SW 오류)과 정확히 매칭 → STA-02(기존 리콜 후 재발 후보)
- HYUNDAI TUCSON 2025 ADAS → 리콜 26V316000 (전방뷰카메라)과 매칭 → RECALLED
- DORMANT 3건 — 리콜 이력은 있으나 최근 6개월 신규 신고 없는 그룹

→ 이게 작업명세서에 명시된 "기각 검증: 진짜 사건(EV9) 결론 도달" 시나리오의 실제 재현. 오늘 이전까진 mock 데이터로만 구조 검증했는데, 이번에 실제 데이터·실제 리콜로 완전히 증명됨.

2. INV 조사 루프 — Gemini 실제 연결해서 완주

`cht01_chat_pipeline.py`를 통해 조사 루프 경로(`investigate_signal` intent)가 실제 Gemini API로 완주됨:

정찰(`get_symptom_distribution`, `get_model_year_breakdown`) 실행
→ Gemini가 가설 생성 → 판정(RETAINED)
→ 판정 근거에 실제 쿼리 결과값 인용 확인 (숫자 지어내지 않음)

- TOOL_REGISTRY 함수 실제 호출, `investigate_ad_hoc()` → `run_investigation_loop()` 전체 사이클 완주 확인

3. 통계 조회 / 안전플래그 / 상태추적 intent

정상 응답 확인 (화재/충돌/부상 집계, 결함 시그널 DB 조회 등).

! 오늘 발견·해결한 이슈 전체 타임라인

이슈	원인	처리
<code>sta_run_pipeline.py</code> import 에러	<code>sta01_02_status_tracking.py</code> 구버전	정윤님 최신본으로 교체
STA 신고 적재 0건 (mock 단계)	mock CSV 컬럼명 불일치	변환 스크립트로 임시 해결
INV-01 조인 실패 (<code>KeyError: odino</code>)	<code>query_templates.py</code> 의 <code>COLUMN_MAP</code> 불일치	값 수정

이슈	원인	처리
조사 결과 대부분 "데이터 없음" (mock 단계)	<code>part_category</code> 표기 불일치	실제 STR 산출물로 교체하며 자연 해소
리콜 매칭 0건 (1차 실제 데이터 시도)	mock 신고 CSV의 odino가 실제 STR 산출물 odino와 안 맞음	Git에서 실제 원본 <code>hk_electrical_recent_full.csv</code> (16,964 건) 발굴·교체
EV9/TUCSON 등 26개 차종이 리콜에서 <code>make=UNKNOWN</code> 으로 새는 버그 발견	<code>sta_recall_loader.py</code> 의 <code>MODEL_TO_MAKE</code> 표에 26개 차종 (EV9, IONIQ 6, 하이브리드/PHEV 변형 등) 누락	효선이 직접 26개 항목 추가, WARN 0건으로 해소
<code>rcl573_components_normalized.csv</code> 에 <code>recall_date</code> 컬럼 자체가 없음	<code>rcl573_parse.py</code> 가 2026-07-13 수정 이전 버전으로 생성한 구버전 파일	리콜 판정 로직 자체엔 영향 없음(안전 폴백 있음), "리콜일 미상"으로만 표시됨 — 정윤님 쪽에서 <code>rcl573_parse.py</code> 재실행 필요

팀에 전달할 것

1. 정윤님

- `MODEL_TO_MAKE` 에 26개 차종(EV9, GV60, IONIQ 6, 각종 HYBRID/PHEV 변형 등) 추가함 — `sta_recall_loader.py` 최신본에 반영 부탁드립니다. (목록은 코드에 주석으로 남겨둠: "2026-07-15 효선 추가")

정윤 답변 :

→ 남겨주신 목록 그대로 `sta_recall_loader.py` 에 반영했습니다. 실제 `rcl573_components_normalized.csv` (198 행)로 직접 돌려서 확인했는데 `recall_records 264건 적재, unmapped_models 0건` — 효선님이 보고하신 수치랑 정확히 일치하는 것까지 확인

- `rcl573_components_normalized.csv` 에 `recall_date` 가 전부 비어있어요 — `rcl573_parse.py` 재실행해서 최신 버전으로 다시 공유 부탁.

정윤 답변 : `rcl573_parse.py` 가 아직 예전(수정 전) 버전이라 `OUT_COLS` 에 `recall_date` 자체가 없었음 다시 정식으로 고침(`verify()` 결측률 체크, `COVERAGE_CSV` 에도 반영).

방식: `rcl573_fill_recall_dates.py` : 지금 있는 CSV 파일 그대로에서 NHTSA 공개 API(캠페인 번호로 조회, 키 불필요)로 76개 캠페인의 실제 리콜일을 바로 채워넣는 스크립트로 수정. 원본 PDF 없이 CSV 파일 하나만 있으면 돼요. 실제로 하나 확인해봤는데(`24V757000` → `2024-10-09`), 실제 EV9 계기판 리콜일과 일치

- 보내주셨던 `recall_crossref.csv` 는 `sta_recall_loader.py` 가 기대하는 스키마와 달라서 이번엔 안 씀 — 이 건 "한·미 시차" 기능(INV-01 `get_us_kr_gap`) 쪽 데이터인지 별도 확인 필요할 것 같아요.

정윤 답변: `recall_crossref.csv` 스키마 수정,

`get_us_kr_gap` 용 데이터, us/kr 리콜일이 한 행에 같이 있는 구조라 두 개의 DataFrame으로 쪼개는 어댑터를 만들어서 연결. "한미 시차"/"국내 리콜 미조치" 질문을 처리하는 새 기능을 CHT-01에 추가하고, 투싼(시차 계산)·싼타페(국내 미조치 감지)·레이(미국 리콜 없음) 3가지 케이스 실제 데이터로 확인

추가로 발견한 버그 :

`query_templates.py` 의 `load_enriched_df()` 가 `FAILDATE` 를 문자열이 아니라 정수로 읽어서, 날짜가 전부 1970년 근처로 깨지는 버그(`get_monthly_trend` 같은 게 조용히 틀린 답을 낼 뻔했음). CHT-01 쪽에서 일단 자동 복구 되게 막아줬는데, 근본적으로는 `raw = pd.read_csv(raw_csv_path, dtype=str)` 로 바꿔주시면 좋을 것 같아요 — 확인 부탁드립니다

2. 주진희님 — 원본 CSV(16,964건, `hk_electrical_recent_full.csv`) Git에서 찾아서 잘 썼어요
3. 상진님 — `llm_struct_test_results.jsonl` + `struct_ev9_202409.jsonl` (총 49건) 실제 STR 산출물로 검증했고, `part_category` 표기(`INSTRUMENT_CLUSTER` 등)가 코드 기대치와 정확히 일치해서 문제없었어요.

다음 할 일 (효선)

- ☐ INV-01/02/03도 이 실제 원본 CSV(`hk_electrical_recent_full_REAL.csv`) + 실제 STR jsonl로 재검증 (오늘은 STA 위주로 검증, INV는 mock 기준 1회만 확인함)
- ☐ 가설 기각 후 재시도, 조건 완화 사다리, LLM의 추가 쿼리 선택 경로 — 아직 실제로 못 본 경로들, 실데이터로 재검증
- ☐ 한·미 시차(`get_us_kr_gap`), 리콜 대조(`check_recall_match`) — `kr_us_gap_v2.csv` 등 Git에 있는 파일로 검증 시도

한 줄 결론

본인 파트(INV-01 쿼리 함수 + INV-02/03 조사 루프)는 실제 Gemini API 연결까지 확인됐고, STA 트랙은 실제 데이터·실제 리콜로 STA-02/RECALLED/DORMANT 상태를 EV9 실사례로 완전히 실증함. 오늘 초반 mock 데이터 기반 검증에서 나온 이슈는 전부 실제 데이터로 교체하며 해소됐고, 로직 자체의 결함은 발견되지 않음. 남은 건 INV 쪽을 동일한 실데이터로 재검증하는 것.

INV-01/02/03 실데이터 검증 — 2026-07-18 (`debug_investigation.py`)

검증 방법

`cht01_chat_pipeline.py` 를 거치지 않고, `investigate_ad_hoc()` 을 직접 호출해서 조사 루프 내부(정찰 결과 → 가설 → 판정)를 전부 펼쳐서 확인. 실제 Gemini API + 실제 데이터(16,964건 원본 CSV + STR 49건) 사용.

질문 조건: {model: EV9, model_year: 2024, symptom: 계기판, part_category: INSTRUMENT_CLUSTER}

✅ 확인된 것

0단계 정찰 — 실제 숫자 산출

```
get_symptom_distribution: INSTRUMENT_CLUSTER 카테고리 총 51건, 증상 60여 종 세분화  
("계기판 랜덤 블랙아웃" 7건, "계기판 블랙" 6건 최다)  
get_model_year_breakdown: EV9 2024년식 51/51건 (100% 집중, share_pct: 100.0)
```

1단계 — Gemini가 정찰 숫자만 보고 가설 생성

"EV9 2024년식 차량에서 총 51건의 신고... '계기판 랜덤 블랙아웃'(7건)과 '계기판 블랙'(6건)이 가장 높은 빈도로 나타나 집중적으로 발생"

→ `symptom_counts` 딕셔너리 값과 정확히 일치. 숫자 지어내지 않음.

3단계 — 판정 근거에 정찰 결과 값 다중 인용

"'계기판 미작동'(4건), '계기판 대부분 미표시'(4건), '계기판 화면 랜덤 블랙아웃'(3건)... 가설을 명확히 뒷받침"

→ `is_retained: True`, 판정 근거 전부 실제 쿼리 결과에서 인용.

결론: 정찰 → 가설 → 판정 전 과정이 실제 데이터·실제 LLM으로 의도한 대로 작동하는 것을 처음으로 완전히 확인함.

! 오늘 검증 중 해결한 문제 (참고용 기록)

증상	원인	해결
<code>get_model_year_breakdown</code> IndexError	groupby 결과 0건 케이스 방어 부족	<code>len(cross)==0</code> 체크 추가
<code>TypeError:</code> <code>get_model_distribution() missing part_category</code>	LLM이 params에 <code>part_category</code> 안 채워도 코드가 기본값 강제 안 함	<code>investigation_loop_v2.py</code> 에서 <code>part_category</code> 항상 보장하도록 수정
<code>model_counts: {}</code> (28건인데 전부 빈 값)	<code>RAW_CSV_PATH</code> 가 160건짜리 옛날 mock을 가리키고 있어서 odino 조인 실패	실제 16,964건 CSV 경로로 재수정

🔍 발견했던 설계 한계 → ✅ 오늘 안에 해결 완료

문제: `get_symptom_distribution` 이 `part_category` 로만 필터링하고 `model / model_year` 로는 안 좁혀짐 (위 EV9 케이스는 우연히 카테고리 전체가 단일 차종이라 문제가 안 드러났던 것).

해결: 기존 함수를 고치는 대신 새 함수 `get_symptom_distribution_by_model` 을 추가하고 `TOOL_REGISTRY` 에 9번째 도구로 등록. LLM이 필요할 때 스스로 선택해서 특정 차종으로 좁혀 재조회하도록 설계 (정찰 단계 자체는 안 건드릴 — "카테고리 전체 큰 그림 먼저" 원칙 유지, `select_query_or_conclude` 단계에서 LLM이 자율적으로 좁혀 씬).

🎯 실전 검증 — 여러 차종 섞인 카테고리(ADAS)로 재테스트

질문 조건: {model: TUCSON, model_year: 2025, symptom: 카메라, part_category: ADAS}

정찰 결과 — 이번엔 진짜로 7개 차종이 섞인 카테고리

```
get_model_year_breakdown breakdown:
  IONIQ 5, K4, PALISADE, SANTA FE, SORENTO, TUCSON, TUCSON HYBRID (총 21건)
top_combo: TUCSON 2025, 4건, share_pct 19.0%
```

1단계 가설 — 근거 약한 "우연의 일치" 추론

"전방충돌방지 허위 작동으로 급제동 증상이 TUCSON 2025년식 모델에 집중되는 경향이 있습니다."

이건 사실 `get_symptom_distribution` 의 최다 증상(4건)과 `get_model_year_breakdown` 의 최다 조합(TUCSON 2025, 4건) 숫자가 우연히 똑같이 4라서 상관관계를 추측한 것 — 두 함수가 서로 교차 계산 안 된 상태에서 나온 느슨한 추론이었음.

2단계 — Gemini가 새 함수를 스스로 선택해서 검증

```
get_symptom_distribution_by_model(model=TUCSON, model_year=2025)
→ TUCSON 2025 4건 중 "전방충돌방지 허위 작동" 증상은 2건뿐
```

→ 새 함수가 처음으로 LLM에 의해 실제 선택되고 정상 작동함을 확인.

3단계 — Gemini가 스스로 1단계 가설을 의심하고 판정을 낮춤

"해당 증상 4건 중 절반만 TUCSON 2025년식에서 발생하여 특정 모델에 집중된다고 판단하기에는 근거가 부족합니다." → HOLD

→ 1단계의 근거 약한 가설이 그대로 최종 결론이 됐다면 오탐(false positive)이었을 것을, 2·3단계가 실제로 걸러낸 사례. "증거 부족 → 보류" 원칙이 실전에서 작동하는 것을 확인.

같은 세션에서 추가로 잡은 버그 → 일반화된 해결로 마무리

증상	원인
<code>TypeError: check_recall_match() got multiple values for argument 'model'</code>	<code>check_recall_match / get_us_kr_gap</code> 이 <code>df</code> 를 아예 안 받는 다른 시그니처(<code>model, ..., recalls_df</code>)인데 다른 함수들과 동일하게 호출됨
<code>TypeError: get_severity_breakdown() got an unexpected keyword argument 'model'</code>	함수마다 받는 파라미터가 제각각인데 LLM이 넘긴 <code>params</code> 를 그대로 다 전달함

해결 — 함수명 하드코딩 대신 일반화: `safe_call_tool()` 헬퍼를 추가해 `inspect.signature()` 로 각 함수가 실제로 받는 파라미터를 코드가 직접 확인 후 자동으로 걸러서 호출하도록 변경. `df` 를 안 받는 함수는 `not_available` 로 정직하게 처리. 함수가 추가/변경돼도 이 로직은 안 건드려도 되는 구조로 정리 완료.

종합 (최종)

INV-01/02/03의 핵심 로직(정찰 강제 → LLM 가설 → 쿼리 선택 → 판정 → 필요시 자율적으로 좁혀서 재검증)이 여러 차종이 섞인 실제 데이터로 완전히 검증됨. 오늘 발견한 설계 한계 하나(모델별 필터 부재)는 새 함수 추가로 당일 해결했고, 그 해결이 실제로 오탐을 막아내는 것까지 실증했음. 파라미터 처리 버그도 일반화된 방식으로 재발 방지 처리 완료.

STA/CHT 트랙 정리 (담당: 정윤)

상태 추적(STA) + 상담사 챗 연동(CHT) 트랙 작업 정리 문서.

이번 라운드는 새 기능 개발이 아니라, 상진님 STR이 v4로 전면 개편되면서 연쇄적으로 깨진 부분을 찾아 고치고 검증한 작업이다.

최종 갱신: 2026-07-20

0. 트랙 개요

- 담당 기능 ID: STA-01, STA-02, CHT-01
- 역할: 결함 시그널 5단계(신규 → 증가 → 리콜 → 잠잠 → 재발) 상태 추적 DB 구축 + 사용자 질문을 조회·검수·답변으로 연결하는 채팅 파이프라인

- **이번 작업의 시작점:** 상진님이 STR을 v2/v3(`part_category` 8종 enum, `odino` 단위)에서 v4(`compdesc1` / `compdesc2` , `cmplid` 단위)로 전면 개편 → 그 스키마를 참조하던 STA/CHT 코드가 여러 지점에서 깨져있는 걸 발견 → 순서대로 찾아 고침
- **관련 파일:** `cht01_chat_pipeline.py` · `sta01_02_status_tracking.py` · `sta_signal_state.py` · `sta_run_pipeline.py` · `sta_recall_loader.py`

1. 진행 현황 (한눈에)

#	문제	심각도	상태
1	STR-05 <code>extract_slots()</code> 튜플 언패킹 누락	높음(연동 시 즉시 에러)	✅ 수정·검증 완료
2	<code>MODEL_ALIASES</code> 27종 중 13종만 반영(EV9 등 15종 누락)	높음(핵심 실종 차종 인식 실패)	✅ 수정·검증 완료
3	STR v4 전환 — CHT-01 자체 로직	중간(부품 카테고리 필터 무효화)	✅ 수정·검증 완료
4	STR v4 전환 — STA 핵심 스키마 (<code>sta01_02_status_tracking.py</code>)	매우 높음 (리콜 매칭 영구 무력화 + 데이터 유실)	✅ 수정·검증 완료
5	STR v4 전환 — <code>sta_signal_state.py</code>	높음(1건 연쇄)	✅ 수정·검증 완료
6	전체 체인 통합 검증	-	✅ 실제 데이터로 완료

2. 문제별 상세

문제 1 — STR-05 연동 시 튜플 언패킹 버그

발견 경위

상진님의 STR-05(`str05_query_understanding.py`)가 처음엔 GitHub 레포에 아예 없어서 찾는 과정을 거쳤고(→ `feature/신구조조화(STR)` 브랜치에 머지 전 상태로 남아있던 것 확인 후 그 브랜치에서 확보), 코드 레벨로 문서와 실물을 대조 검증했다.

발견한 문제

실제 `extract_slots()` 는 `dict` 하나가 아니라 (갱신된 `slots`, 결과 `dict`) **튜플**을 반환하는데, `cht01_chat_pipeline.py` 의 `extract_slots_safe()` 가 이 튜플을 언패킹하지 않고 그대로 돌려주고 있었다. 이 상태로 연동하면 `chat()` 내부의 `slots.get("상태")` 호출에서 `AttributeError` 가 발생한다.

추가로, 단순히 언패킹만 고치는 걸로는 부족하다는 걸 발견했다 — 반환되는 두 값 중 ****result** (화면 표시용, 완료 시 3필드)**를 그대로 다음 턴 `slot_state` 로 저장하고 있었는데, 무진전 카운터(`_no_progress` , "2회 연속 진전 없으면 대화 포기" 로직에 쓰이는 값)는 **내부용 slots**에만 있고 **result** 엔 없다. 즉 언패킹만 고쳤다면 무진전 카운터가 매 턴 유실돼 그 안전장치가 실질적으로 한 번도 작동하지 않았을 것이다.

수정 내용

- `extract_slots_safe()` 가 (`updated_slots` , `result`) 를 명시적으로 분리해서 반환하도록 수정
- `chat()` 에서 `updated_slot_state` (다음 턴 `slot_state` 로 저장, 무진전 카운터 보존용)와 `slot_result` (답변·확인 문장 생성용)를 분리해서 각각 올바른 용도로만 사용하도록 재작성

문제 2 — 차종 인식 목록(`MODEL_ALIASES`) 불일치

발견 경위

STR-05의 차종 registry(`CAR_MODELS` , 27종)와 CHT-01의 `MODEL_ALIASES` 를 대조 검증하는 과정에서 발견.

발견한 문제

CHT-01의 `MODEL_ALIASES` 엔 13종밖에 없었고, **EV9(프로젝트의 대표 실증 사례 — 실제 리콜 24V757000 매칭 사례로 쓰인 차종)를 포함해 15종이 누락**돼 있었다. 이 상태에서 STR-05가 차종을 정확히 뽑아도, `MODEL_EN_TO_KO` 역매핑을 쓰는 `_us_kr_gap_lookup()` (한·미 시차 조회) 같은 기능에서 한국어 변환이 안 돼 조용히 "데이터 없음"으로 새는 문제가 있었다. 반대로 `RAY` 는 CHT-01에만 있고 STR-05엔 없는 반대 방향 불일치도 확인.

수정 내용

`MODEL_ALIASES` 를 STR-05의 27종 전부 커버하도록 확장(한국어 별칭 추가). 접두어 충돌 방지를 위해 `NIRO EV` 를 `NIRO` 보다 먼저 배치.

문제 3, 4, 5 — STR v4 전환 대응 (가장 큰 작업)

배경

상진님이 STR을 v4로 전면 개편한 이유: ① LLM이 원본에 이미 있는 `COMPDESC` (부품 분류)를 무시하고 자체 판단 (`part_category`)하면서 정확도가 떨어졌던 문제, ② 하나의 신고(`ODINO`)에 여러 결함 유형이 섞여 있을 때 분류가 애매했던 문제 — NHTSA 원본이 이미 `CMPLID` (행) 단위로 분리해줬다는 걸 발견하고, 처리 단위를 `ODINO` → `CMPLID` 로, 분류 체계를 `part_category` (8종 enum) → `compdesc1` / `compdesc2` (원본 `COMPDESC` 그대로, 7종)로 바꿈.

3단계로 나눠 대응

① CHT-01 자체 소유 로직 (1차)

`part_category` 참조 29곳을 전수 확인해서, CHT-01이 온전히 소유한 로직(`parse_question` , `_evidence_samples_from_df` , `_scope_text` 등)은 즉시 `compdesc1` 기준으로 교체. INV-01(호선님 담당)로 나가는 파라미터는 키 이름 ("`part_category`")은 INV 쪽 함수 계약이라 그대로 두되, 값만 `compdesc1` 기준으로 소싱해서 INV가 나중에 업데이트되면 바로 맞물리게 준비. STA DB 관련 2곳은 스키마 파일을 아직 못 받아 이 시점엔 보류.

② STA 핵심 스키마 (`sta01_02_status_tracking.py`) — 가장 심각한 문제 발견

`_v4` 라는 이름이 붙은 파일을 받아 검토했는데, **실제로는 v4가 전혀 반영 안 된 v2/v3 시절 파일**이었다. 구체적으로 3가지 문제:

1. `load_structured_jsonl()` 이 여전히 `part_category` 를 읽음 — v4 json엔 이 필드가 없어서 항상 `None` 으로 적재됨
2. `structured_results` / `complaint_reports` 가 `odino` 를 PK로 사용 — 한 `odino` 가 여러 `cmplid` 로 쪼개지는 v4 데이터를 넣으면 `ON CONFLICT(odino) DO UPDATE` 가 이전 `cmplid` 행을 계속 덮어써서 **odino당 딱 1개 행만 살아남는 심각한 데이터 유실**
3. `_component_matches()` 가 `part_category=None` 을 받아 항상 `False` 반환 — **리콜 매칭이 영구적으로 무력화**돼, STA-02(재발) 판정과 `defect_signals` 의 RECALLED/DORMANT/RECURRING 3개 상태가 **절대 나올 수 없는 상태**였음(에러 없이 조용히 항상 "신규"로만 판정)

재설계 내용:

- `structured_results` PK를 `odino` → `cmplid` 로 변경, `compdesc1` / `compdesc2` 컬럼 신규 추가
- `part_category` 컬럼을 `structured_results` / `defect_status_tracking` / `defect_signals` 전부 `compdesc1` 로 교체
- `PART_RECALL_KEYWORDS` (구 8종)를 v4 실제 7종 카테고리 기준으로 재구성(상진님 문서의 실측 `compdesc2` 하위분류명 기반)
- `classify_and_track()` 을 "한 `odino`에 딸린 여러 `cmplid` 행 중 하나라도 리콜 매칭되면 그 신고 전체를 STA-02로 본다"는 로직으로 재설계

③ sta_signal_state.py + CHT-01 마무리 (2차)

fetch_signal_groups() 등 같은 컬럼을 참조하던 부분을 compdesc1 기준으로 맞춤. STA 스키마가 확정된 뒤 CHT-01에서 보류해뒀던 2곳(_query_status_db(), 답변 조립부의 top.get())도 마저 compdesc1으로 정리.

3. 검증 방법과 결과

수정할 때마다 문법 체크(py_compile)만으로 끝내지 않고, 실제 STR v4 산출물(str01_sample100_v4_results.jsonl, 100건)로 실행까지 확인했다.

검증 항목	결과
v4 100건 적재 시 데이터 유실 여부	100/100 유실 없음 확인 (다중 CMPLID인 ODINO 11486092의 3개 cmplid 행 전부 생존)
compdesc1 분포	상진님 리포트 수치와 정확히 일치 (UNKNOWN OR OTHER 45 / ELECTRICAL SYSTEM 23 등)
리콜 매칭(STA-02) 정상 작동 여부	가짜 리콜 등록 후 classify_and_track() 실행 → STA-02 정상 판정 (수정 전엔 영구 불가능했던 것)
5단계 시그널(defect_signals)	recompute_all_signal_states() 엔드투엔드 실행 → 정상 저장·조회 확인
전체 체인 통합 실행	합성 원본 CSV + 합성 RCL573 CSV + 실제 v4 100건으로 sta_run_pipeline.run() 을 처음부터 끝까지(적재 → 리콜적재 → 분류 → 시그널계산 → export) 실행 → STA-01 82 / STA-02 2, 시그널 NEW 18 / RECURRING 1 / RISING 1로 정상 산출, RECURRING 판정도 실제 리콜과 매칭된 정확한 사례에서 나옴
CHT-01의 STA 조회 함수(_query_status_db)	실제로 생성된 DB에 {model: TUCSON, compdesc1: FORWARD COLLISION AVOIDANCE} 조건으로 직접 질의 → 정확한 시그널(RECURRING, 2건) 반환 확인

4. 최종 산출물

파일	변경 여부
cht01_chat_pipeline.py	실질 수정 (슬롯 언패킹, 차종 27종, STR v4 대응)
sta01_02_status_tracking.py	실질 수정 (스키마 전면 재설계)
sta_signal_state.py	실질 수정 (컬럼명 반영)
sta_run_pipeline.py	검증만, 변경 없음
sta_recall_loader.py	검증만, 변경 없음(part_category 미참조라 v4 전환과 무관)

5. 알려진 한계 / 다음 단계

- **INV-01(효선님) 미반영**: TOOL_REGISTRY 함수들이 아직 part_category 기준이라, CHT-01이 값은 compdesc1 기준으로 정확히 넘겨도 INV-01 쪽에서 실제 필터링까지는 안 됨. 효선님이 현재 INV-01을 v4에 맞춰 수정 작업 중이며, 완료되면 완전히 맞물릴 예정.
- **COMPDESC1_RECALL_KEYWORDS** : 리콜 CSV의 component 텍스트와 문자열 겹침만 보는 휴리스틱이라, "CRUISE CONTROL" 과 "ADAPTIVE CRUISE CONTROL" 처럼 서로 다른 실제 하위분류가 부분적으로 겹치는 경우가 있음(구버전에도 있던 동일한 성격의 한계). 팀 검수 권장.
- **mentions_existing_recall 필드**: 상진님이 STR-04에서 만들었던 이 필드는, STA-01/02가 이미 실제 리콜DB 대조(공식 데이터 기준)로 유사 기능을 하고 있다고 판단해 STR v4 스키마에서 제외하기로 결정됨. 다만 "아직 STA 리콜DB에 반영 안 된 최신 리콜을 신고자가 스스로 언급하는 경우"엔 STA가 못 잡는 신호를 잡을 수 있어, 교차 검증용으로는 여전히 논의 가치가 있다고 상진님과 공유함.

- 기존에 이미 만들어진 `.db` 파일이 있다면 새 스키마(PK 구조 자체 변경)와 안 맞아 마이그레이션이 아니라 **재 생성이 필요함**(`init_db()` 부터 다시).